NATIONAL UNIVERSITY OF SINGAPORE

SCHOOL OF COMPUTING
FINAL ASSESSMENT FOR
Semester 2 AY2024/2025

CS2030S Programming Methodology II

April 2025 Time Allowed 120 minutes

INSTRUCTIONS TO CANDIDATES

1. This assessment paper contains 24 questions and comprises 16 printed pages, including this page.
2. Write all your answers in the answer sheet provided.
3. The total marks for this assessment is 100. Answer **ALL** questions.
4. This is a **CLOSED BOOK** assessment. You are only allowed to refer to a single double-sided A4 cheatsheet.
5. All programs in this assessment paper use Java 21 and are compiled with the flags `-Xlint:unchecked` and `-Xlint:rawtypes`.

Multiple Choice Questions (60 points)

For each question below, **shade your answer in the corresponding bubble on the answer sheet.** Each question is worth 3 points. If multiple answers are equally appropriate, pick one and shade the chosen answer on the answer sheet. Do NOT shade more than one answer. If none of the answers is appropriate, shade X in the answer sheet.

1. (3 points) Consider the following Java classes `Car` and `Engine`.

```
public class Car {
    private Engine engine;

    // Implementation Omitted

    public void start() {
        engine.startEngine();
    }
}

public class Engine {
    // Implementation Omitted
}
```

What best describes the relationship between `Car` and `Engine`?

   A. Inheritance
   B. Composition
   C. Polymorphism
   D. Immutability
   E. Encapsulation

Shade X on the answer sheet if none of the choices above is correct.

2. (3 points) Which of the following statements about Java's exception handling is correct?

   A. The `finally` block is executed only if an exception is thrown in the `try` block.
   B. A compiler will make sure all unchecked exceptions are handled in a program, or it will not allow a program to compile.
   C. Unchecked exceptions are subclasses of `RuntimeException`.
   D. The `throws` keyword is used to throw an exception from a method.
   E. Custom exceptions in Java must extend the `Error` class.

Shade X on the answer sheet if none of the choices above is correct.

3. (3 points) Consider the following code snippet:

```
Integer a = 50;
int b = a;
```

Which of the following best describes the behaviour demonstrated by this code?

   A. The code demonstrates autoboxing and unboxing.
   B. The code demonstrates a violation of the Liskov Substitution Principle.
   C. The code demonstrates polymorphism.
   D. The code demonstrates the concept of type erasure in Java.
   E. The code results in a `NullPointerException`.

Shade X on the answer sheet if none of the choices above is correct.

4. (3 points) In Java, which of the following statements about interfaces is TRUE?

A. A class can implement multiple interfaces.

B. Interfaces can contain instance variables.

C. Interfaces can be instantiated.

D. An interface can extend multiple classes.

E. An abstract class implementing an interface must override all abstract methods in the interface.

Shade X on the answer sheet if none of the choices above is correct.

Questions 5 to 10 refer to the following classes `Box` and `SubBox`, with `SubBox` <: `Box`. `SubBox` has an inner class `IntBox` that inherits from `SubBox<Integer>`.

```
@FunctionalInterface
interface F<T, R> {
  R run(T x);
}

class Box<T> {
  private T x;

  public Box(T x) {
    this.x = x;
  }

  public T get() {
    return x;
  }
}

class SubBox<T> extends Box<T> {
  public SubBox(T x) {
    super(x);
  }

  public static <T> SubBox<T> of(int x) {
    @SuppressWarnings("unchecked")
    SubBox<T> box = (SubBox<T>) new IntBox(x);
    return box;
  }

  public <U> SubBox<T> merge(F<SubBox<U>, T> f, SubBox<U> other) {
    return this;
  }

  // INNER CLASS
  private static class IntBox extends SubBox<Integer> {
    public IntBox(Integer x) {
      super(x);
    }

    @Override
    public <U> IntBox merge(F<SubBox<U>, Integer> f, SubBox<U> other) {
      return new IntBox(this.get() + f.run(other));
    }
  }
}
```

5. (3 points) Consider the line of code below:

```
SubBox.<String>of(4);
```

Which of the statements below is CORRECT?

A. The line of code leads to a compilation error.
B. The line of code compiles with a warning and runs with a `ClassCastException` exception.
C. The line of code compiles with a warning and runs without any exception.
D. The line of code compiles without any error/warning and runs with a `ClassCastException` exception.
E. The line of code compiles without any error/warning and runs without any exception.

Shade X on the answer sheet if none of the choices above is correct.

6. (3 points) Consider the line of code below:

```
SubBox.<String>of(4).get();
```

Which of the statements below is CORRECT?

A. The line of code leads to a compilation error.
B. The line of code compiles with a warning and runs with a `ClassCastException` exception.
C. The line of code compiles with a warning and runs without any exception.
D. The line of code compiles without any error/warning and runs with a `ClassCastException` exception.
E. The line of code compiles without any error/warning and runs without any exception.

Shade X on the answer sheet if none of the choices above is correct.

7. (3 points) Consider the assignment of the following lambda expression to a variable `f`:

```
f = box -> box.get().intValue();
```

where the instance method `intValue()` is defined in the `Number` class and returns an `int` value. Recall that `Integer` inherits from `Number`.

Which of the following is a valid compile-time type for `f`?

(i) `F<SubBox<Number>, ? extends Number>`
(ii) `F<Box<? super Integer>, ? extends Number>`
(iii) `F<? super Box<Integer>, Integer>`

A. (ii) only
B. (iii) only
C. (i) and (ii) only
D. (i) and (iii) only
E. (i), (ii), and (iii) only

Shade X on the answer sheet if none of the choices above is correct.

8. (3 points) Consider the expression below:

```
SubBox.<Integer>of(5).merge(x -> 0, new SubBox<>("hello"));
```

What is the compile-time type of the return value of the expression?

A. This is a trick question. The expression does not compile.
B. `IntBox`
C. `SubBox<Integer>`
D. `SubBox<String>`
E. `SubBox<Object>`

Shade X on the answer sheet if none of the choices above is correct.

9. (3 points) Consider the expression below (which is the same as the previous question):

```
SubBox.<Integer>of(5).merge(x -> 0, new SubBox<>("hello"));
```

What is the inferred type of `U`?

A. No valid `U` can be inferred. The expression does not compile.

B. `String`

C. `Integer`

D. `Object`

E. `SubBox<String>`

Shade X on the answer sheet if none of the choices above is correct.

10. (3 points) Consider the expression below (which is the same as the previous question):

```
SubBox.<Integer>of(5).merge(x -> 0, new SubBox<>("hello"));
```

What is the run-time type of the value returned by the expression?

A. This is a trick question. The expression does not compile.

B. `SubBox<T>`

C. `SubBox<Integer>`

D. `IntBox`

E. `String`

Shade X on the answer sheet if none of the choices above is correct.

11. (3 points) Consider the Java assignment statement below:

```
a = b;
```

We denote the compile time type of `a` and `b` as CTT(`a`) and CTT(`b`), respectively. The run-time type of `b` is denoted as RTT(`b`).

Which of the following statement(s) correctly describe the Java's type system?

(i) CTT(`b`) must be a subtype of CTT(`a`)

(ii) RTT(`b`) must be a subtype of CTT(`a`)

(iii) RTT(`b`) must be a subtype of CTT(`b`)

A. (i) only

B. (iii) only

C. (i) and (ii) only

D. (ii) and (iii) only

E. (i), (ii), and (iii)

Shade X on the answer sheet if none of the choices above is correct.

Questions 12 and 13 refer to the following class `CF`. The method `randomWaitTime()` (code not shown) takes an undeterministic amount of time to complete.

```
import java.util.concurrent.CompletableFuture;
import java.util.Random;

public class CF {
  public static void main(String[] args) {
    CompletableFuture<Integer> future1 = CompletableFuture.supplyAsync(() -> {
      randomWaitTime();
      System.out.print("P");
```

```
      return 10;
    });

    CompletableFuture<Integer> future2 = CompletableFuture.supplyAsync(() -> {
      randomWaitTime();
      System.out.print("Q");
      return 20;
    });

    CompletableFuture<Integer> future3 = future1.thenCombine(future2, (x, y) -> x + y)
      .thenApply(result -> { System.out.print("R"); return result * 2; });

    future3.join();
    CompletableFuture<Integer> finalResult = future3.thenApplyAsync(result -> {
      randomWaitTime();
      System.out.print("S");
      return result + 10;
    });

    System.out.print("T");
  }
}
```

12. (3 points) What value would be computed by the `CompletableFuture` `finalResult` if it is completed?

    A. 10

    B. 20

    C. 60

    D. 70

    E. 80

    Shade X on the answer sheet if none of the choices above is correct.

13. (3 points) Given the above program, which of the following statements are true?

    (i) P is always printed before Q

    (ii) P is always printed before R

    (iii) Q is always printed before R

    (iv) R is always printed

    (v) S is always printed

    (vi) T is always printed

    A. (i), (ii), (iv), and (vi) only

    B. (ii), (iii), (iv), and (vi) only

    C. (i), (iii), (iv), (v), and (vi) only

    D. (ii), (iii), (iv), (v), and (vi) only

    E. (i), (ii), (iii), (iv), (v), and (vi) only

    Shade X on the answer sheet if none of the choices above is correct.

For Question 14 to Question 16 you have to evaluate a stream pipeline. For your reference, we have given you some helpful Stream methods in Appendix A.

14. (3 points) Consider the stream below:

```
Stream
  .iterate(1, x -> x + 1)
  .map(i -> Stream.iterate(1, x -> x <= i, x -> x + 1))
  .limit(5)
  .distinct()
  .count()
```

    Does the stream pipeline correctly evaluate to a value, and if so, what is the value of evaluating this stream pipeline?

    A. Yes, it evaluates to 2

    B. Yes, it evaluates to 3

    C. Yes, it evaluates to 5

    D. No, there is a compilation error.

    E. No, there is a run-time error.

    Shade X on the answer sheet if none of the choices above is correct.

15. (3 points) Consider the stream below:

```
Stream
  .iterate(1, x -> x + 1)
  .flatMap(i -> Stream.iterate(1, x -> x <= i, x -> x + 1))
  .limit(5)
  .distinct()
  .count()
```

    Does the stream pipeline correctly evaluate to a value, and if so, what is the value of evaluating this stream pipeline?

    A. Yes, it evaluates to 2

    B. Yes, it evaluates to 3

    C. Yes, it evaluates to 5

    D. No, there is a compilation error.

    E. No, there is a run-time error.

    Shade X on the answer sheet if none of the choices above is correct.

16. (3 points) Consider the stream below:

```
Stream
  .iterate(1, x -> x + 1)
  .limit(5)
  .map(i -> Stream.iterate(1, x -> x <= i, x -> x + 1))
  .filter(x -> x.count() % 2 == 0)
  .flatMap(x -> x)
  .count()
```

Does the stream pipeline correctly evaluate to a value, and if so, what is the value of evaluating this stream pipeline?

A. Yes, it evaluates to 2

B. Yes, it evaluates to 3

C. Yes, it evaluates to 5

D. No, there is a compilation error.

E. No, there is a run-time error.

Shade X on the answer sheet if none of the choices above is correct.

Questions 17 to 20 refer to the following class `Lazy<T>`.

```
public class Lazy<T> {
    private Producer<T> producer;
    private T value;
    private boolean isEvaluated;

    private Lazy(Producer<T> producer) {
        this.producer = producer;
        this.isEvaluated = false;
    }

    public static <T> Lazy<T> of(Producer<T> producer) {
        return new Lazy<>(producer);
    }

    public T get() {
        if (!isEvaluated) {
            value = producer.produce();
            isEvaluated = true;
        }
        return value;
    }

    public void reset() {
        this.isEvaluated = false;
        this.value = null;
    }
}
```

17. (3 points) Consider the following code excerpt:

```
Lazy<Integer> lazyValue = Lazy.of(() -> {
  System.out.println("Computing value...");
  return 42;
});

for (int i = 0; i < 4; i++) {
  lazyValue.get();
}
```

How many times is "Computing value..." printed?

A. 0

B. 1

C. 2

D. 3

E. 4

Shade X on the answer sheet if none of the choices above is correct.

18. (3 points) Now, consider the following code excerpt:

```
Lazy<Integer> lazyValue = Lazy.of(() -> {
  System.out.println("Computing value...");
  return 42;
});

for (int i = 0; i < 4; i++) {
  lazyValue.get();
  if (i % 2 == 0) {
    lazyValue.reset();
  }
}
```

How many times is "Computing value..." printed?

A. 0

B. 1

C. 2

D. 3

E. 4

Shade X on the answer sheet if none of the choices above is correct.

Questions 19 and 20 also refer to the following class `A`.

```
public class A {
  private int x;

  public A(int x) {
    this.x = x;
  }

  public int getX() {
    return x;
  }

  public void setX(int x) {
    this.x = x;
  }
}
```

19. (3 points) Now consider the following class `Test`:

```
class Test {
  public static void main(String[] args) {
    A a = new A(10);
    Lazy<Integer> lazyValue = Lazy.of(() -> a.getX() * 2);
    System.out.print(lazyValue.get() + " ");
```

```
    a.setX(20);
    System.out.print(lazyValue.get());
  }
}
```

If the class compiles, explain what it will print.

A. 10 20

B. 20 20

C. 20 30

D. 30 30

E. The code does not compile as `a` is not effectively final.

Shade X on the answer sheet if none of the choices above is correct.

20. (3 points) Now consider the following revised class `Test`:

```
class Test {
  public static void main(String[] args) {
    A a = new A(10);
    Lazy<Integer> lazyValue = Lazy.of(() -> a.getX() * 2);
    System.out.print(lazyValue.get() + " ");
    a = new A(20);
    System.out.print(lazyValue.get());
  }
}
```

If the class compiles, explain what it will print.

A. 10 20

B. 20 20

C. 20 30

D. 30 30

E. The code does not compile as `a` is not effectively final.

Shade X on the answer sheet if none of the choices above is correct.

Short Answer Questions (40 points)

21. (12 points) **Monad**

Consider the following implementation of `ParityMonad`. The intention of this class is to track an integer value along with a boolean flag that flips each time a transformation is applied. The `flatMap` method updates the value while toggling the flag, allowing the class to record whether an odd or even number of transformations have occurred. Note that ^ is the XOR operator in Java.

```
interface Transformer<S, T> {
  T transform(S s);
}

class ParityMonad {
  private int value;
  private boolean flag;

  public ParityMonad(int value, boolean flag) {
    this.value = value;
    this.flag = flag;
  }

  public static ParityMonad of(int value) {
    return new ParityMonad(value, true);
  }

  public ParityMonad flatMap(Transformer<Integer, ParityMonad> map) {
    ParityMonad temp = map.transform(this.value);
    return new ParityMonad(temp.value, temp.flag ^ this.flag);
  }

  public String toString() {
    return "[" + this.value + ", " + this.flag + "]";
  }
}
```

Test if `ParityMonad` is a Monad by checking if it obeys monadic laws. Use the following functions.

```
Transformer<Integer, ParityMonad> t1 = x -> new ParityMonad(x + 1, true);
Transformer<Integer, ParityMonad> t2 = x -> new ParityMonad(x - 1, false);
```

(a) (4 points) Does `ParityMonad` obey the left identity law?

If it does not, demonstrate it by filling in the following blanks. Use the string representation of `ParityMonad` as returned by its `toString` method to describe the return values.
If it does obey the law, write in "NA" in all the blanks below.

`ParityMonad.of(2).flatMap(_____);` returns _____

`_____.transform(2);` returns _____

(b) (4 points) Does `ParityMonad` obey the right identity law?

If it does not, demonstrate it by filling in the following blanks. Use the string representation of `ParityMonad` as returned by its `toString` method to describe the return values.
If it does obey the law, write in "NA" in all the blanks below.

`ParityMonad monad = ParityMonad.of(2).flatMap(t1);`

`monad.flatMap(_____);` returns _____

`monad;` returns _____

(c) (4 points) Does `ParityMonad` obey the associative law?

If it does not, demonstrate it by filling in the following blanks. Use the string representation of `ParityMonad` as returned by its `toString` method to describe the return values.

If it does obey the law, write in "NA" in all the blanks below.

`ParityMonad.of(2).flatMap(_____).flatMap(_____);` returns _____

`ParityMonad.of(2).flatMap(_____);` returns _____

22. (7 points) **Maybe**

Consider the following method `getUserProfile`.

Study the method below:

```
Maybe<UserProfile> getUserProfile(Database db, String userId) {
  if (db == null || userId == null) {
    return Maybe.none();
  }

  Maybe<User> maybeUser = db.getUserById(userId);
  User user;
  if (maybeUser.equals(Maybe.none())) {
    return Maybe.none();
  } else {
    user = maybeUser.get(); // cannot call get()
  }

  Maybe<Address> maybeAddress = user.getAddress();
  Address address;
  if (maybeAddress.equals(Maybe.none())) {
    return Maybe.none();
  } else {
    address = maybeAddress.get(); // cannot call get()
  }

  return Maybe.of(new UserProfile(user, address));
}
```

Rewrite the method using `Maybe<T>` such that:

- it consists of only a single return statement;
- it does not use additional external classes or methods beyond those already used in the given code below;
- must not use `null` or `get`;
- it does not contain `if`, the ternary `? :` operators, or other branching logic besides those internally provided by `Maybe<T>`.

The implementation details of the external classes `Address`, `Database`, and `User` used in the method are not required to answer this question.

For your reference, we have given you some helpful `Maybe` methods in Appendix B.

Fill in the following template for your answer.

```
Maybe<UserProfile> getUserProfile(Database db, String userId) {
    return Maybe.of(_____________________________________________________)
        ._____(_____________________________________________________)
        ._____(_____________________________________________________);
}
```

23. (11 points) **InfiniteList**

Consider the following implementation of `InfiniteList<T>`.

```
class InfiniteList<T> {
  private final Producer<T> head;
  private final Producer<InfiniteList<T>> tail;

  public static <T> InfiniteList<T> iterate(T init1, T init2,
                                            Transformer<T, Transformer<T, T>> next) {
    return new InfiniteList<T>(
        () -> init1,
        () -> InfiniteList.iterate(init2, next.transform(init1).transform(init2), next));
  }

  public InfiniteList(Producer<T> head, Producer<InfiniteList<T>> tail) {
    this.head = head;
    this.tail = tail;
  }

  private T head() {
    return this.head.produce();
  }

  private InfiniteList<T> tail() {
    return this.tail.produce();
  }

  public <R> InfiniteList<R> map(Transformer<T, R> mapper) {
    return new InfiniteList<>(
        () -> mapper.transform(this.head()),
        () -> this.tail().map(mapper));
  }

  public T get(int n) {
    return n == 0 ? this.head() : this.tail().get(n - 1);
  }
}
```

(a) (4 points) Study the factory method `iterate` and the code snippet below.

```
InfiniteList<Integer> list = InfiniteList.iterate(1, 3, x -> y -> x + y);
for (int i = 0; i < 4; i++) {
  int element = list.get(i);
  System.out.println(element);
}
```

Write down the numbers printed by the code snippet above.

(b) (7 points) Study the code snippet below.

```
interface F extends Transformer<Integer,Integer> { }

F f0 = x -> x + 1;
F f1 = x -> 2 * x;
InfiniteList<F> l = InfiniteList.iterate(f0, f1,
    f -> g -> (x -> f.transform(g.transform(x))));

for (int i = 0; i < 4; i++) {
  System.out.println(l.get(i).transform(1));
}
```

Write down the numbers printed by the code snippet above.

24. (10 points) **Stack and Heap Diagram**

This question refers to the `InfiniteList<T>` class in Question 23.

Draw the content of the stack and heap in the given answer sheet when the program runs and reaches Line A using the convention used in class. Label the stack frames with the method name, the objects on the heap with the class name, and the variables/fields with their names and their values. Use ϕ for uninitialized variables.

```
class Ones {
  public static InfiniteList<Integer> ones() {
    return new InfiniteList<>(() -> 1, () -> ones());
  }

  public static void main(String[] args) {
    args = null;
    InfiniteList<Integer> i1 = ones();
    InfiniteList<String> i2 = i1.map(x -> x.toString());
    // Line A
  }
}
```

Appendix A: Stream<T> API (Abridged)

filter

```
Stream<T> filter(Predicate<? super T> predicate)
```

Returns a stream consisting of the elements of this stream that match the given predicate.

map

```
<R> Stream<R> map(Function<? super T,? extends R> mapper)
```

Returns a stream consisting of the results of applying the given function to the elements of this stream.

flatMap

```
<R> Stream<R> flatMap(Function<? super T,? extends Stream<? extends R>> mapper)
```

Returns a stream consisting of the results of replacing each element of this stream with the contents of a mapped stream produced by applying the provided mapping function to each element. Each mapped stream is closed after its contents have been placed into this stream. (If a mapped stream is null an empty stream is used, instead.)

limit

```
Stream<T> limit(long maxSize)
```

Returns a stream consisting of the elements of this stream, truncated to be no longer than maxSize in length.

sorted

```
Stream<T> sorted(Comparator<? super T> comparator)
```

Returns a stream consisting of the elements of this stream, sorted according to the comparator.

count

```
long count()
```

Returns the count of elements in this stream.

forEach

```
void forEach(Consumer<? super T> action)
```

Performs an action for each element of this stream.

distinct

```
Stream<T> distinct()
```

Returns a stream consisting of the distinct elements of this stream.

of

```
static <T> Stream<T> of(T... values)
```

Returns a sequential ordered stream whose elements are the specified values.

iterate

```
static <T> Stream<T> iterate(T seed, UnaryOperator<T> f)
```

Returns an infinite sequential ordered Stream produced by iterative application of a function f to an initial element seed, producing a Stream consisting of seed, f(seed), f(f(seed)), etc.

iterate

```
static <T> Stream<T> iterate(T seed, Predicate<? super T> p, UnaryOperator<T> f)
```

Returns an infinite sequential ordered Stream produced by iterative application of a function f to an initial element seed, producing a Stream consisting of seed, f(seed), f(f(seed)), etc. The stream terminates as soon as the given predicate `p` returns false.

Appendix B: Maybe<T> API (Abridged)

of

```
static <T> Maybe<T> of(T t)
```

Create an instance of Maybe containing the value t if it is is not null or an instance of Maybe with missing value otherwise.

map

```
<U> Maybe<U> map(Transformer<? super T, ? extends U> transformer)
```

If the value within this Maybe is missing, just return the current instance; otherwise, apply the given transformation on the value and return a Maybe with the new value intact.

flatMap

```
<U> Maybe<U> flatMap(Transformer<? super T, ? extends Maybe<? extends U>> transformer)
```

If the value within this Maybe is missing, just return the current instance; otherwise, apply the given transformation on the value and return a Maybe with the new value intact. This is similar to map except that the given transformation returns a Maybe and flatMap does not wrap the new value with an additional layer of Maybe.

filter

```
Maybe<T> filter(BooleanCondition<? super T> cond)
```

If the value within this Maybe does not exist, just return the current instance. Otherwise, check the value against the given condition and return a Maybe with missing value (if cond test is false), or return a Maybe with the value intact.

orElse

```
T orElse(T t)
```

If the value within this Maybe is missing, return the given t; otherwise, return the value in this Maybe.

OrElseGet

```
T orElseGet(Producer<? extends T> producer)
```

If the value within this Maybe is missing, return the value produced by the given producer. Otherwise, return the value in this Maybe.

ifPresent

```
void ifPresent(Consumer<? super T> consumer)
```

If the value within this Maybe is missing, does nothing; Otherwise, consume the value with the given consumer.

END OF PAPER